需求-模型追溯模块开发手册

面向 Codex 等编码智能体的实现规范

基于当前项目中的 XMI 信息提取源码与追溯关系实验主流程编写

生成日期：2026 年 7 月 3 日

交付物：PDF 开发手册 + trace_system 环境 requirements.txt

手册目标与适用边界

本文用于指导 Codex、Cursor Agent、Copilot Agent 等编码智能体，在当前项目源码基础上开发“XMI 输入驱动的需求-模型追溯系统”。本手册不是用户操作手册，而是面向开发实施的工程约束、模块划分、接口约定、数据流设计和验收标准。

本次目标功能包括：

- 用户输入从实验 CSV 数据集改为 XMI 文件。
- 支持用户指定 XMI 文件中的子系统，并将 XMI 内容解析为当前实验流程可消费的数据集格式。
- 知识图谱改为动态加载：若已构建完成，后续运行直接加载缓存，不再重复构建；若没有缓存，检查用户是否上传了知识图谱相关数据。
- 最终生成的全部追溯关系以可视化方式展示到前端，由用户审核、修正；审核完成后，可将确认后的追溯关系更新到知识图谱。
- 增加前端界面，包括 XMI 文件上传入口、可选知识图谱上传入口、子系统选择入口和追溯关系审核界面。

硬性约束

- 保留当前项目已有功能语义，优先通过适配层、服务层和配置层接入新能力，避免对已有实验函数做大规模重写。
- 以当前源码为事实来源。编码智能体开始修改前，必须重新阅读相关文件，而不是只按本文猜测实现。
- 将“XMI 文件”作为主输入。CSV 只作为中间数据格式或兼容模式，不应再作为主业务入口。
- 将知识图谱加载、构建、缓存、更新封装成独立服务，不能继续把固定路径和重建逻辑散落在主流程中。
- 对所有 LLM 输出进行结构化校验，保留当前两阶段判定与 JSON 归一化思路。
- 区分“LLM 自动判定结果”和“人工审核确认结果”，只有审核通过或人工修正后的结果才写回长期知识图谱。
- 保留原始输入、解析结果、模型输出、审核操作和知识图谱更新日志，便于复现和追责。
- 将系统运行环境名称统一设置为 trace_system。用户电脑无法使用 conda 时，优先使用 Python venv 或 uv 管理环境；所有部署命令、README、requirements 说明和启动脚本示例都必须使用该环境名。
- 开发前必须先用 venv 按当前已有环境文件 output pdf requirements.txt 搭建 trace_system 环境，并完成依赖安装验证。不得在未搭好环境的情况下开始改造业务代码。
- 编码智能体必须严格按照本 PDF 开发手册实施。若实现中需要偏离本文的架构、路径、环境、测试或交付要求，必须先记录原因并获得用户确认。
- 将开发主路径设置为 TRACE_SYSTEM。其中 The_experiment_of_trace_relation 是主要追溯框架，XMI message 是 XMI 信息提取与生成相关文件，test-XMI 是测试 XMI 文件目录。
- 当前阶段 XMI 信息提取与生成步骤暂不加入 LLM 辅助生成相关信息。LLM 只应保留在追溯相关信息构建和追溯关系智能判定等已有规划环节中。
- 完成开发后必须进行全链路测试，覆盖从 XMI 上传、XMI 解析、候选样本构建、KG 动态加载、追溯相关信息构建、LLM 判定、前端审核到知识图谱写回的完整流程。

非目标

- 本手册不要求替换现有大模型服务、向量模型或本体推理器。

- 本手册不要求重新设计知识图谱本体结构，只要求兼容当前实验知识图谱格式并增加缓存、审核写回和元数据管理。
- 本手册不要求实现统计 F1 指标。当前验收指标以 Accuracy 和关系类型识别准确率为主。

当前项目能力基线

编码智能体开发前，应先阅读并确认以下源码边界。文件名以当前项目为准，如后续目录调整，应以实际源码位置为准。

现有模块 | 应复用的能力与注意事项

main.py | 当前实验主流程入口。负责读取固定 CSV、构建 KG、构造节点对、解析端口信息、调用本体推理、检索上下文、构造 LLM 输入、调用两阶段 LLM 并保存结果。后续应把它拆成可配置 orchestration，而不是继续写死数据路径。

Port_Information/information_extract.py | XMI 信息提取能力。已有 XMIInformationExtractor，可按包路径、系统名、锚点和 hop 范围提取类、端口、连接器、信息流。应作为 XMI 接入层核心。

Port_Information/test_port_information.py | 对简化文本中的 Port/Connector/Information Flow 做解析，形成结构化端口与信息流 JSON。可继续作为 XMI 解析后的补充结构化步骤。

kg_retrieve/read_sysml_raw_data.py | 当前实验数据集到节点、边、候选对、子系统节点/边的构造逻辑。新增 XMI 适配器应尽量输出它可以消费的字段结构。

kg_retrieve/construct_KG.py | 当前知识图谱构建入口。现状是每次从 CSV 构建 NetworkX 图。后续应在外层增加 KGManager，支持缓存加载和按需构建。

kg_retrieve/use_rules.py 与 kg_retrieve/Retreive_Rules.py | 当前追溯相关证据检索逻辑，包括同类节点、同子系统上下文、Top-K 历史案例检索。不得宣传已有“空结果兜底”，除非后续显式实现。

enlarge_function/Get_Model_Function.py | 对模型元素进行功能描述扩写，生成或补充 contextA。该能力属于“追溯相关信息构建”模块。

protege_reasoner/use_reasoner.py | 本体规则推理入口，负责构造推理输入并调用 Java 推理器，返回候选关系集合。该能力也属于“追溯相关信息构建”模块。

call_llm/call_llm.py | 两阶段 LLM 调用、JSON 修复与结果归一化。应继续作为追溯关系智能判定核心。

add_information_to_graph/add_information.py | 将 LLM 判定结果转为图边结构。后续写回知识图谱时应扩展审核字段，而不是直接把未审核结果作为长期事实。

目标系统总体架构

目标系统建议采用“前端 + API 服务 + 追溯任务编排 + 可缓存知识图谱 + 审核写回”的结构。编码智能体应尽量在当前项目中增加薄服务层，不要把 UI、文件上传、知识图谱缓存、LLM 调用全部堆叠到一个脚本里。

层级 | 职责

前端交互层 | 提供 XMI 上传、知识图谱上传、子系统选择、任务启动、进度展示、追溯结果图谱/表格审核、人工修正和写回确认。

API 服务层 | 接收文件和参数，创建项目/任务，管理任务状态，返回解析结果、追溯结果、审核状态和图谱可视化数据。

XMI 接入与适配层 | 调用现有 XMIInformationExtractor，按用户指定子系统解析 XMI，并转换成当前实验主流程可消费的数据集记录。

知识图谱管理层 | 负责 KG 缓存加载、缺失检查、上传 KG 数据校验、按当前实验格式构建 KG、保存元数据、审核后增量更新。

追溯相关信息构建层 | 负责模型元素功能描述扩写、Top-K 相似案例检索、同子系统上下文提取、本体规则推理候选关系生成。

智能判定层 | 基于模型元素、需求、上下文、证据和候选关系调用两阶段 LLM，输出结构化追溯关系判定结果。

审核与写回层 | 将最终追溯关系展示给用户审核；接受、拒绝、修改和新增边；仅将确认后的关系写回知识图谱并记录审计日志。

建议新增模块与责任划分

以下文件名是建议方案，编码智能体可以根据项目实际框架调整，但职责边界应保持清晰。

建议模块 | 核心职责

services/xmi_dataset_adapter.py | 封装 XMI 文件解析、子系统过滤、信息流提取、需求/模型候选记录生成。输出应兼容 read_sysml_raw_data.py 的字段约定。

services/kg_manager.py | 封装知识图谱缓存加载、构建、保存、元数据校验、上传 KG 数据检查、审核结果增量写回。

services/trace_context_builder.py | 封装功能描述扩写、检索证据、本体规则推理和 LLM 输入上下文组装。

services/trace_runner.py | 编排单个项目/任务的端到端追溯流程，替代当前 main.py 中写死路径的实验入口。

services/review_service.py | 管理追溯结果审核状态、人工修改记录、写回 KG 前的校验和审计日志。

api/routes.py | 提供上传、任务运行、结果查询、审核修改和 KG 写回接口。

frontend/ | 提供文件上传页和追溯关系审核页。若项目已有前端框架，应遵循既有框架；若没有，可采用轻量 HTML/JS 或 Streamlit/FastAPI 模板。

核心功能开发规范

XMI 文件作为主输入

前端与 API 应将 XMI 文件作为主输入。用户上传后，系统应保存原始文件，并为每次上传生成 project id 或 run id。解析时应支持以下参数：

- XMI 文件路径或上传文件对象。
- 用户指定的子系统名称列表。
- 可选的包路径，例如当前代码中的 1 Problem Domain / 2 White Box / 2 Conceptual Subsystems Communication。
- 可选锚点与 hop 范围，用于围绕子系统收缩解析范围。
- 输出目录，用于保存解析后的中间文本、JSON 和候选数据集。

开发要求：

- 复用 XMIInformationExtractor 的系统筛选能力，不要重新手写一套 XMI 正则解析。
- XMI 信息提取与 XMI 信息生成相关实现以 XMImessage 路径下的内容为参考，但当前阶段不得在该步骤中增加 LLM 辅助生成逻辑。
- XMI 解析结果应转换成当前主流程可以处理的记录，包括模型元素、需求元素、所属子系统、端口/连接器/信息流和候选节点对。
- 如果 XMI 中缺少需求元素，应返回明确错误或空候选提示，而不是让后续 LLM 流程静默失败。
- 用户指定子系统后，候选对构造应优先限制在该子系统及其相关连接范围内。
- 保存解析元数据：原始文件名、文件 hash、解析时间、指定子系统、包路径、解析出的元素数量和候选对数量。

XMI 到当前实验数据集格式的适配

当前实验流程依赖 CSV/表格型记录。新增适配器应把 XMI 解析结果转换为“类 CSV 的内部数据结构”，以便继续复用 build_node_pairs、add_nodes_from_pairs、update_subsystem_nodes、update_subsystem_edges 等逻辑。

建议的内部记录字段：

字段 | 含义

model_name | 模型元素名称，例如 Block、Action、Port、ValueProperty 等。

model_type | 模型元素类型，应映射到当前 TYPE_PREFIX 支持的类型或新增兼容类型。

model_context | 模型元素描述，可来自 XMI 原始注释、属性，也可由 LLM 扩写后写入 contextA。

requirement_name | 需求元素名称。

requirement_type | 需求类型，例如 Functional Requirement、Design Requirement、Performance Requirement 等。

requirement_context | 需求文本、说明或约束。

subsystem | 所属子系统，用于候选裁剪和同子系统上下文提取。

ports/connectors/flows | 与该候选对相关的端口、连接器和信息流摘要。

如果现有函数要求的字段名与上表不同，编码智能体应以 read_sysml_raw_data.py 的实际读取逻辑为准，并在适配器中做字段映射。

知识图谱动态加载

知识图谱管理是本次改造重点。当前实验逻辑每次从 CSV 重新构建 KG；目标系统应改为“先加载缓存，缺失再构建，仍缺失则检查用户上传 KG 数据”。

推荐流程：

```
load_or_build_kg(project_id, kg_upload=None, force_rebuild=False):
1. 根据 project_id、知识图谱源文件 hash、构建版本生成 cache_key
2. 若 force_rebuild=False 且缓存存在：
读取图对象和 metadata，校验格式版本，直接返回 Graph
3. 若缓存不存在：
检查用户是否上传 KG CSV/库文件/历史图谱包
4. 若用户上传了 KG：
校验字段格式，调用 construct_kg_init 或兼容构建函数
保存 Graph 缓存和 metadata
5. 若未上传 KG：
返回可解释错误，提示前端要求用户上传 KG 数据或选择空图初始化策略
```

缓存建议：

- 图对象缓存：Data/KnowledgeGraph/<project_id>/kg.graphml 或 kg.pkl。
- 元数据：Data/KnowledgeGraph/<project_id>/metadata.json。
- 源文件副本：Data/KnowledgeGraph/<project_id>/source/。
- 审核写回日志：Data/KnowledgeGraph/<project_id>/review_updates.jsonl。

元数据至少包含：

- project_id
- source_type: uploaded / generated / cached
- source_hash
- schema_version
- built_at
- node_count
- edge_count
- reviewed_edge_count

注意事项：

- 不要改变当前实验 KG 的核心节点、边属性含义。新增审核字段应兼容旧图，而不是破坏旧图。
- 缓存命中时不应调用 CSV 重建函数。
- 如果未来实现“空图初始化”，必须在 UI 和日志中明确标识，避免把无知识图谱条件下的结果误认为基于历史知识。
- 当前检索代码没有明确实现“空结果兜底处理”。若产品需要该能力，必须单独新增逻辑和测试，不得只在文档中宣称支持。

追溯相关信息构建

该模块负责给 LLM 判定提供结构化输入，包含模型元素功能扩写、证据检索、同子系统上下文和本体规则推理。

必须包含的子功能：

- 模型元素功能描述扩写：复用 enlarge_element 和 generate_description 思路，为缺少 contextA 的模型元素生成可读功能描述。扩写结果应缓存，避免同一元素反复调用 LLM。
- 相似案例检索：复用 retrieve 及相关规则，检索 Top-K 历史追溯案例和同类型节点上下文。
- 同子系统上下文提取：围绕候选模型元素和需求元素提取所属子系统、相邻节点和信息流上下文。
- 本体规则推理：复用 build_reasoner_input 和 reasoner，生成候选关系集合，为 LLM 限定可选关系范围。
- LLM 输入组装：调用 construct_inf_json 或等价逻辑，把模型、需求、上下文、证据、候选关系整合为统一 JSON。

输出对象建议为：

```
TraceContext = {
"project_id": "...",
```

"pair_id": "...",
"model_node": {...},
"requirement_node": {...},
"subsystem_context": [...],
"retrieved_cases": [...],
"candidate_relations": [...],
"model_function_expansion": "...",
"timing": {...}
}

追溯关系智能判定

智能判定层继续复用当前两阶段 LLM 逻辑：

- 第一阶段：判断需求与模型元素之间是否存在追溯关系。
- 第二阶段：在存在关系时，基于候选关系集合选择具体关系类型。

结构化输出至少包含：

- exists: 是否存在追溯关系。
- selected_relation: 关系类型。
- confidence: 置信度。
- reasoning_process: 简要推理过程。
- reason: 判定理由。
- evidence_ids: 使用的证据编号。
- candidate_relations: 本体推理给出的候选关系集合。

不要新增 F1 作为强制验收指标。性能验收建议：

- Accuracy 不低于 64%。
- 关系类型识别准确率不低于 56%。
- 常规规模下，单个候选对端到端处理耗时不超过约 2.5 分钟；该指标包含 LLM 调用，需记录超时样本。

追溯关系可视化与人工审核

系统应把所有最终追溯关系展示到前端，供用户审核。展示方式应同时支持图和表：

- 图视图：需求节点、模型节点、子系统节点和追溯关系边。边颜色或样式表示待审核、已通过、已拒绝、已修改。
- 表视图：逐条展示模型元素、需求元素、关系类型、置信度、LLM 理由、证据摘要、审核状态和人工备注。
- 详情面板：展示完整上下文、Top-K 案例、本体候选关系、LLM 原始 JSON 和解析后的规范结果。

审核操作必须支持：

- 接受关系。
- 拒绝关系。
- 修改关系类型。
- 修改置信度或人工可信等级。
- 添加人工备注。
- 新增遗漏关系。
- 删除错误边。
- 批量保存审核结果。

审核状态建议：

状态 | 含义
pending | LLM 已生成，用户尚未审核。默认状态。
accepted | 用户确认该关系正确，可以写回 KG。
rejected | 用户确认该关系错误，不写回 KG，但保留审计记录。
edited | 用户修改了关系类型、端点或说明，修改后的版本可写回 KG。
added | 用户手工新增的关系，可写回 KG。

审核结果写回知识图谱

写回 KG 时只处理 accepted、edited 和 added 状态的关系。pending 和 rejected 不应进入长期知识图谱。

新增或更新边时，建议补充以下属性：

- relation: 关系类型。
- Source_type: HumanReviewed 或 LLMReviewed。
- review_status: accepted / edited / added。
- reviewer: 审核用户。
- reviewed_at: 审核时间。
- llm_confidence: 原始 LLM 置信度。
- human_comment: 人工备注。
- evidence_ids: 证据编号。
- run_id: 追溯任务编号。

写回规则：

- 如果边不存在，新增边。
- 如果边已存在且关系类型一致，更新审核元数据和证据。
- 如果边已存在但关系类型不同，应保留历史记录，并由用户确认覆盖、并存或标记冲突。
- 每次写回前先生成 preview，让用户看到将新增、修改和跳过的边。
- 写回后保存新 KG 缓存，并更新 metadata 中的边数量和审核边数量。

前端界面开发规范

上传与任务配置界面

上传界面应作为第一屏核心功能，不需要营销式首页。界面至少包含：

- XMI 文件上传控件，限制扩展名为 .xmi 或 .xml。
- 子系统选择输入。可以先允许用户手动输入系统名；若解析后能列出系统，应支持多选。
- 可选知识图谱上传控件，支持当前实验 KG CSV 或打包后的 KG 缓存。
- 包路径、锚点、hop 范围等高级参数折叠区。
- “开始解析/追溯”按钮。
- 任务状态展示：等待、解析中、构建 KG、检索上下文、LLM 判定、待审核、完成、失败。

追溯关系审核界面

审核界面应以“表格 + 图谱 + 详情抽屉/面板”的组合为主：

- 左侧或上方：关系表格，支持按状态、关系类型、置信度、子系统过滤。
- 中央：图谱可视化，节点和边可以点击，高亮当前关系。
- 右侧或底部：详情面板，展示证据、LLM 理由、本体候选关系和人工审核控件。
- 顶部：批量接受、批量拒绝、保存审核、写回知识图谱按钮。

可视化建议：

- 如果项目已有 sysml_kg.html 或 vis-network/PyVis 页面，可先复用其图谱展示方式。
- 节点颜色按类型区分：需求、模型元素、子系统、端口/连接器。
- 边颜色按审核状态区分：待审核、接受、拒绝、修改、新增。
- 大图场景必须分页或按子系统过滤，避免一次渲染过多节点导致前端卡死。

API 与数据接口建议

若使用 FastAPI 或类似后端框架，建议提供以下接口。实际实现可调整路径，但语义应保持一致。

方法 | 路径 | 用途

POST | /api/projects | 创建项目或运行空间，返回 project_id。

POST | /api/projects/id/xmi | 上传 XMI 文件并保存原始文件。

POST | /api/projects/id/kg | 上传可选 KG 数据或 KG 缓存包。

POST | /api/projects/id/trace/run | 启动解析、KG 加载、追溯关系生成任务。

GET | /api/projects/id/trace/status | 查询任务状态和阶段性日志。

GET | /api/projects/id/trace/results | 查询追溯关系结果列表。

GET | /api/projects/id/trace/graph | 获取前端图谱可视化所需 nodes/edges JSON。

PATCH | /api/projects/id/trace/results/edge_id | 修改单条关系的审核状态、关系类型、备注等。

POST | /api/projects/id/trace/review/batch | 批量审核关系。

POST | /api/projects/id/kg/update-preview | 生成写回 KG 的预览。

POST | /api/projects/id/kg/update | 将已审核关系写回知识图谱。

追溯结果对象

```
TracePrediction = {
"edge_id": "...",
"run_id": "...",
"model_node_id": "...",
"model_name": "...",
"requirement_node_id": "...",
"requirement_name": "...",
"relation": "...",
"exists": true,
"confidence": 0.82,
"reason": "...",
"reasoning_process": "...",
"candidate_relations": ["...", "..."],
"evidence": [...],
"review_status": "pending",
"review_relation": null,
"human_comment": "",
"updated_at": "..."
}
```

图谱可视化对象

```
TraceGraph = {
"nodes": [
{"id": "...", "label": "...", "type": "Requirement", "subsystem": "..."}
],
"edges": [
{"id": "...", "source": "...", "target": "...",
```

```
"relation": "...", "status": "pending", "confidence": 0.82}
```

```
]
```

```
}
```

任务编排建议

端到端运行流程建议如下：

- 前端上传 XMI，用户填写或选择子系统。
- API 保存文件，计算 hash，创建项目和运行编号。
- XMI 适配层解析 XMI，生成中间 JSON 和候选数据集。
- KGManager 根据 project id 和 KG 源数据加载缓存；没有缓存则检查用户上传 KG；可构建时构建并缓存，不可构建时返回前端提示。
- 对候选对逐条构建追溯相关信息：模型功能扩写、同子系统上下文、Top-K 案例、本体候选关系。
- 调用两阶段 LLM，生成结构化追溯关系结果。
- 保存原始输出、规范化结果、耗时和错误日志。
- 前端展示表格和图谱，用户审核。
- 用户确认写回预览。
- KGManager 将已审核关系增量写回知识图谱，保存新缓存和审计日志。

配置与路径要求

需要把当前硬编码路径迁移到配置中。建议增加统一配置项：

开发主路径固定为 TRACE_SYSTEM。编码智能体应按以下路径职责组织和说明系统：

- TRACE_SYSTEM：最终系统开发主路径，用于整合前端、API、追溯任务编排、配置和交付物。
- The_experiment_of_trace_relation：主要追溯框架，包含当前需求-模型追溯实验主流程、知识图谱构建/检索、本体推理、LLM 判定和结果分析代码。
- XMImessage：XMI 信息提取与生成相关文件，当前阶段作为 XMI 解析适配参考，不加入 LLM 辅助生成相关信息。
- test-XMI：测试用 XMI 文件目录，性能测试和回归测试应优先从该目录读取样例。
- output：最终说明文件、测试报告和环境配置文件输出目录。完成开发后，README.md、requirements.txt、性能测试报告等交付文件应汇总到该目录下。

运行环境名称固定为 trace_system。当前已有环境依赖文件位于 output pdf requirements.txt；编码智能体开始开发前必须先基于该文件搭建可运行环境。如果用户电脑无法使用 conda，编码智能体应优先使用 Python 自带 venv 管理环境：

```
cd TRACE_SYSTEM
```

```
py -3.12 -m venv .venv
```

```
.\.venv\Scripts\Activate.ps1
```

```
python -m pip install --upgrade pip
```

```
pip install -r output\pdf\requirements.txt
```

```
python -c "import pandas, networkx, openai, fastapi"
```

如果 PowerShell 禁止激活脚本，可提示用户执行：

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

如果使用 uv，可采用：

```
cd TRACE_SYSTEM
```

```
uv venv .venv --python 3.12
```

```
.\.venv\Scripts\Activate.ps1
```

```
uv pip install -r output\pdf\requirements.txt
```

```
python -c "import pandas, networkx, openai, fastapi"
```

完成所有开发和验证后，必须把最终环境依赖导出并合并到 output/requirements.txt。若使用 venv，可执行：

```
python -m pip freeze > output\requirements.freeze.txt
```

随后编码智能体应人工合并依赖：保留系统真正需要的运行依赖到 output/requirements.txt，避免把调试工具、临时 notebook、无关训练框架或本机私有路径全部写入正式依赖文件。

配置项 | 含义

DATA_ROOT | 项目数据根目录。

UPLOAD_ROOT | 用户上传 XMI 和 KG 数据保存目录。

KG_CACHE_ROOT | KG 缓存与元数据保存目录。

RESULT_ROOT | 追溯结果、审核结果、错误日志保存目录。

XMI_DEFAULT_PACKAGE_PATH | 默认 XMI 包路径。

EMBEDDING_MODEL_PATH | SentenceTransformer 模型路径。

REASONER_JAR_PATH | 本体推理器 jar 路径。

LLM_BASE_URL / LLM_MODEL | LLM 服务配置。

MAX_RETRIEVAL_K | Top-K 检索数量。

ENABLE_MODEL_EXPANSION_CACHE | 是否启用模型元素描述扩写缓存。

编码智能体不得继续把 Data/GenerateContext/Air_Condition/KG.csv 之类实验路径作为唯一入口。可保留 demo 配置或兼容脚本，但生产入口必须由用户上传和配置驱动。

测试与验收标准

功能验收

测试项 | 通过标准

XMI 上传解析 | 上传合法 XMI 后，系统能生成解析元数据、子系统元素清单和候选数据集。

子系统范围过滤 | 指定一个或多个子系统后，候选对和上下文范围能按子系统收缩。

KG 缓存命中 | 同一项目第二次运行时，KGManager 直接加载缓存，不重新调用构建函数。

KG 缓存缺失检查 | 无缓存时，系统检查用户是否上传 KG 数据；未上传时给出明确前端提示。

模型描述扩写 | 缺少 contextA 的模型元素能生成或读取缓存的功能描述。

本体候选关系 | 每个候选对能记录本体推理候选关系；推理失败时记录错误，不导致整个任务崩溃。

LLM 判定结果 | 输出包含存在性、关系类型、置信度、理由和证据引用。

审核界面 | 用户可在表格和图谱中查看、接受、拒绝、修改、新增追溯关系。

KG 写回 | 仅 accepted、edited、added 关系写回 KG；rejected 和 pending 不写回。

审计日志 | 每次审核和写回都有 run id、用户、时间、原始结果和修改结果。

性能指标

指标设置应偏工程可落地，不宜过紧。下列数值是最低验收门槛，不是优化上限；编码智能体开发时应尽量接近或优于原始目标。测试报告应同时记录最低门槛和优化目标：达到优化目标记为 PASS_TARGET，未达到优化目标但达到最低门槛记为 PASS_MINIMUM，低于最低门槛记为 FAIL。正式验收时不得存在 FAIL；若某项因缺少测试输入被标记为 SKIP，应补齐对应样例、日志或审核结果后再验收。

- XMI 文件规模：支持常规约 2.4—8 MB XMI 文件上传与解析。
- XMI 解析耗时：常规规模下不超过约 38 秒。
- XMI 解析成功率：常规模型解析成功率不低于 72%。
- 候选样本字段完整率：主要字段完整率不低于 72%。
- KG 缓存加载：缓存命中时不超过约 13 秒。
- KG 首次构建：常规实验规模下不超过约 2.5 分钟；超出时必须展示阶段进度。
- 追溯相关信息构建：不包含 LLM 扩写时，单个候选对不超过约 38 秒。
- 智能判定：常规规模下单个候选对端到端不超过约 2.5 分钟，包含 LLM 调用。
- 质量指标：Accuracy 不低于 64%，关系类型识别准确率不低于 56%。
- 前端审核：约 800 条以内关系应支持筛选和分页审核；图谱过大时按子系统或关系状态分层展示。

性能指标测试脚本

本手册配套提供 output/pdf/trace_system_performance_tests.py 作为性能指标验收脚本。后续开发时可将该脚本移动到项目的 tests/ 或 tools/ 目录，但不得删除其中的最低门槛和优化目标判定逻辑。

该脚本覆盖以下指标：

- XMI 文件规模、解析成功率和解析耗时。
- 候选样本字段完整率。
- KG 缓存加载耗时和首次构建耗时。
- 追溯相关信息构建耗时。
- 单样本端到端智能判定耗时。

- 追溯关系存在性 Accuracy。
- 关系类型识别准确率。
- 可视化审核结果规模和审核状态记录。

示例命令如下：

```
.\.venv\Scripts\Activate.ps1
python output/pdf/trace_system_performance_tests.py ^
--project-root TRACE_SYSTEM ^
--xmi-dir test-XMI ^
--candidate-csv Data/GenerateContext/Air_Condition/Judge.csv ^
--kg-csv Data/GenerateContext/Air_Condition/KG.csv ^
--kg-cache-path Data/KnowledgeGraph/demo/kg.pkl ^
--timing-json temporary_results/<run_id>/timing.json ^
--metrics-json result/<experiment>/metrics.json ^
--review-results-json result/<run_id>/review_results.json ^
--report-json output/pdf/performance_metrics_report.json
```

脚本输出每个指标的实际值、最低门槛、优化目标和状态。后续系统完成开发后，编码智能体必须在 README 中说明该脚本的运行方式，并在交付物中保留最近一次测试报告。

编码智能体执行清单

编码智能体实施时按以下顺序推进：

- 严格阅读本 PDF 开发手册，并将其中路径、环境、模块、测试和交付要求作为本次开发约束。
- 开发前先创建并激活 trace_system 的 venv 环境，使用 output pdf requirements.txt 安装当前已有依赖，并完成基本 import 验证。
- 重新阅读 main.py、XMI 提取模块、KG 构建/检索模块、LLM 调用模块和图写回模块。
- 确认并记录路径职责：TRACE_SYSTEM 为最终开发主路径，The_experiment_of_trace_relation 为追溯框架，XMImessage 为 XMI 信息提取与生成参考，test-XMI 为测试 XMI 数据目录。
- 增加配置层，把固定路径、模型路径、reasoner 路径、结果目录迁移到配置。
- 新增 XMI 到内部候选数据集的适配器，并用至少一个真实 XMI 文件做解析验证。
- 当前阶段 XMI 信息提取与生成步骤不得新增 LLM 辅助生成相关信息；如需补充模型功能描述，应放在追溯相关信息构建阶段处理。
- 新增 KGManager，实现缓存命中、上传 KG 检查、首次构建、元数据保存和缓存加载。
- 抽取 trace runner，复用现有检索、本体推理、LLM 调用和结果保存逻辑。
- 新增审核数据模型，给每条追溯关系分配稳定 edge id 和 review status。

- 增加 API 接口，完成上传、运行、状态查询、结果查询、审核修改和 KG 写回。
- 增加前端上传页面和审核页面。
- 增加测试：XMI 解析、KG 缓存、追溯任务、审核写回、异常场景。
- 增加并运行性能指标测试脚本，覆盖 XMI 解析、候选样本、KG 加载/构建、追溯相关信息构建、智能判定准确率、端到端耗时和审核规模。
- 最后必须运行全链路测试：XMI 上传与解析、子系统范围选择、候选样本生成、KG 动态加载或构建、追溯相关信息构建、追溯关系智能判定、前端可视化审核、审核结果写回 KG。测试完成后保存截图、日志和输出文件。
- 完成开发后，必须编写或更新项目根目录 README.md，说明整个系统的安装、配置、启动、上传 XMI、选择子系统、审核追溯关系和写回知识图谱的完整使用方法。
- 完成开发后，必须把最终说明文件、环境配置文件和测试报告汇总输出到 output 文件夹，并把最终依赖合并到 output/requirements.txt。

风险与处理建议

风险 | 处理建议

XMI 结构与实验 CSV 字段不一致 | 通过适配器做字段映射；缺失字段显式记录为空或由 LLM 扩写，不要让下游函数隐式失败。

知识图谱缓存与源数据不一致 | 用 source hash、schema version 和 built at 校验；不一致时提示用户选择重建或继续使用旧缓存。

LLM 输出不稳定 | 继续使用 JSON 修复与归一化；保留原始输出；失败样本进入错误队列，允许重试。

本体推理器路径硬编码 | 将 jar 路径纳入配置，并在启动时做可用性检查。

图谱太大导致前端卡顿 | 按子系统、关系状态和置信度过滤；表格分页；图谱按当前筛选结果渲染。

未审核结果误写入 KG | 写回接口必须校验 review status；pending/rejected 一律跳过。

用户修改结果后无法追踪来源 | 每条边保留 LLM 原始结果、人工修改结果、用户、时间和 run id。

交付物要求

完成开发后，至少应交付：

- XMI 上传和解析功能。
- KG 动态加载、缓存和上传检查功能。
- 基于当前实验流程的追溯任务运行入口。
- 追溯关系可视化审核界面。
- 审核结果写回 KG 功能。
- 端到端样例数据、运行日志和结果文件。
- 全链路测试报告。报告必须覆盖 XMI 上传与解析、候选样本构建、KG 动态加载、追溯相关信息构建、智能判定、前端审核和 KG 写回。
- 单元测试或最小回归测试说明。
- 性能指标测试脚本和最近一次 performance_metrics_report.json 测试报告；报告中不得存在 FAIL，正式验收不得保留关键指标 SKIP。

- 部署说明和环境依赖文件。最终环境依赖必须合并输出为 output/requirements.txt；如另存完整冻结依赖，可命名为 output/requirements.freeze.txt。
- 项目根目录 README.md。README 必须面向最终使用者，至少包含 trace_system 环境创建、依赖安装、后端启动、前端访问、XMI 上传、子系统选择、追溯任务运行、人工审核、知识图谱写回和常见问题处理。
- output 文件夹下的最终交付汇总，包括 README.md、requirements.txt、性能测试报告和必要的启动说明。

附录：对当前表格口径的建议

若需要把该功能继续填入项目统计表，建议采用以下功能划分：

类别 | 功能 | 功能项 | 性能指标

智能建模辅助 | XMI 模型接入与子系统范围解析 | XMI 上传解析、包路径定位、子系统筛选、端口/连接器/信息流提取、候选数据集生成 | 支持约 2.4—8 MB XMI 文件解析；解析成功率不低于 72%；单文件解析一般不超过约 38 秒

智能建模辅助 | 需求-模型候选样本与知识图谱构建管理 | 候选样本字段生成、子系统范围缩减、基础去重、KG 缓存加载、上传 KG 检查、首次构建和元数据管理 | 主要字段完整率不低于 72%；缓存命中时 KG 加载一般不超过约 13 秒；首次构建一般不超过约 2.5 分钟

智能建模辅助 | 追溯相关信息构建 | 模型功能描述扩写、同子系统上下文、Top-K 案例检索、本体候选关系推理、LLM 输入组装 | 不含 LLM 扩写时单候选对不超过约 38 秒；候选证据数量根据历史样本匹配情况动态返回

智能建模辅助 | 追溯关系智能判定与结果评估 | 两阶段 LLM 判定、关系类型选择、置信度与理由输出、结果统计 | Accuracy 不低于 64%；关系类型识别准确率不低于 56%；单样本端到端推理一般不超过约 2.5 分钟

智能建模辅助 | 追溯关系可视化审核与知识图谱更新 | 图谱/表格展示、人工接受/拒绝/修改/新增、写回预览、KG 更新和审计日志 | 支持约 800 条关系分页审核；写回仅处理已审核关系；审核和写回操作保留日志记录